Querying Knowledge Graphs and Graph Database Solutions

Ernesto Jiménez-Ruiz

Senior Lecturer in Artificial Intelligence

Before we start...

Contact

- **Ernesto Jiménez-Ruiz**
- City St George's, University of London
- Contact:
  - ernesto.jimenez-ruiz@citystgeorges.ac.uk
  - https://ernestojimenezruiz.github.io/

GitHub Repository

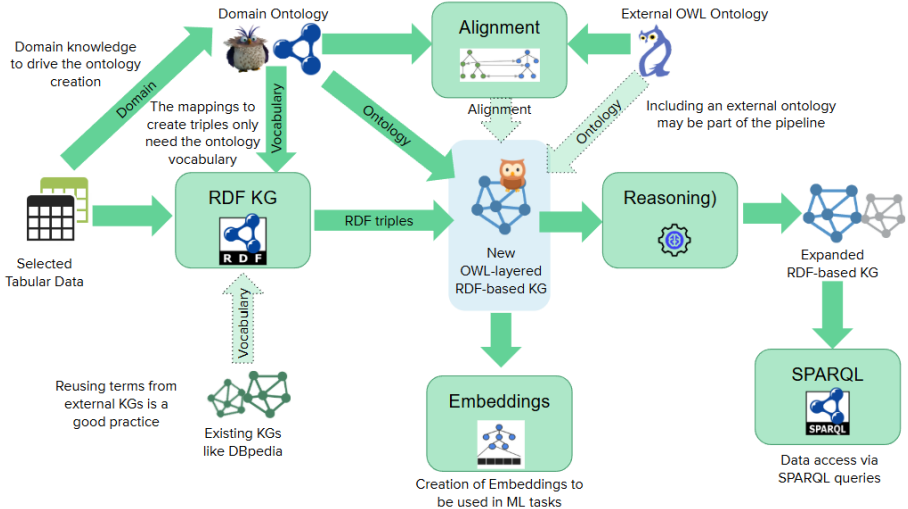
https://github.com/city-knowledge-graphs/kg-course-valgrai

Course Organization

1. Introduction to Knowledge Graphs.
2. Modelling Ontologies with OWL.
3. RDFS Semantics, Reasoning and OWL 2 Profiles.
4. From Tabular Data to Knowledge Graphs.
5. Querying Knowledge Graphs and Graph Database Solutions.
6. Knowledge Graphs and Language Models.

The global picture

The global picture



Learning outcomes for today

- Formulate queries in SPARQL.
- Execute SPARQL queries against a local and a remote repository.
  - Programmatically
  - Visual interfaces (`Protégé`, GraphDB, Web forms)
- Compare and analyze different Graph database solutions.
- Create repositories with GraphDB. Load and explore data with GraphDB.
- Run queries over the SPARQL Endpoints defined by GraphDB.

SPARQL Query Language for RDF-based KGs

SPARQL by Example

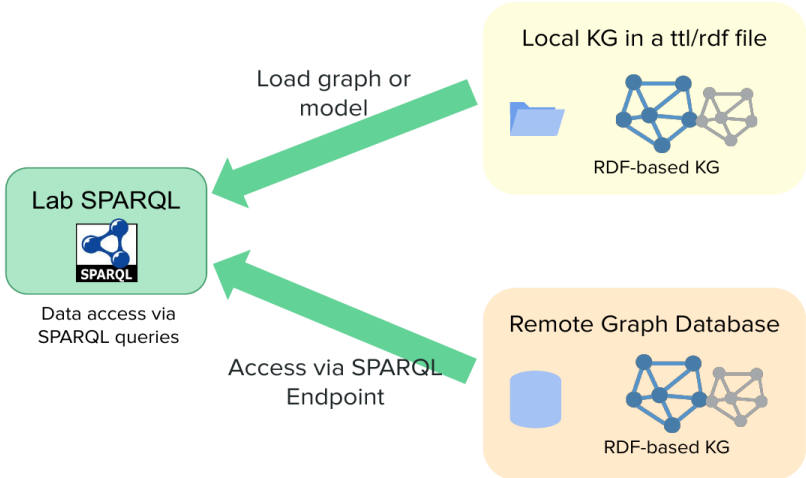
SPARQL

- SPARQL Protocol And RDF Query Language
- **Standard language** to query graph data represented as **RDF triples**
- W3C Recommendations
  - **SPARQL 1.0:** W3C Recommendation 15 January 2008
  - **SPARQL 1.1:** W3C Recommendation 21 March 2013

SPARQL

- SPARQL Protocol And RDF Query Language
- **Standard language** to query graph data represented as **RDF triples**
- W3C Recommendations
  - **SPARQL 1.0:** W3C Recommendation 15 January 2008
  - **SPARQL 1.1:** W3C Recommendation 21 March 2013
- Documentation:
  - Syntax and semantics of the SPARQL query language for RDF:
    `http://www.w3.org/TR/rdf-sparql-query/`
    `https://www.w3.org/TR/sparql11-overview/`
  - Examples: `https://www.w3.org/2008/09/sparql-by-example/`

SPARQL: local and remote KG access



SPARQL Examples (i)

- Based on DBpedia: RDF version of Wikipedia with information about actors, movies, etc.: `https://dbpedia.org/`
- Web interface for SPARQL writing: `http://dbpedia.org/sparql`

SPARQL Examples (i)

- Based on DBpedia: RDF version of Wikipedia with information about actors, movies, etc.: `https://dbpedia.org/`
- Web interface for SPARQL writing: `http://dbpedia.org/sparql`

People called “Johnny Depp”

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
SELECT DISTINCT ?jd WHERE {
    ?jd foaf:name "Johnny Depp"@en .
}
```

SPARQL Examples (i)

- Based on DBpedia: RDF version of Wikipedia with information about actors, movies, etc.: `https://dbpedia.org/`
- Web interface for SPARQL writing: `http://dbpedia.org/sparql`

People called "Johnny Depp"

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
SELECT DISTINCT ?jd WHERE {
    ?jd foaf:name "Johnny Depp"@en .
}
```

Answer:

?jd
<http://dbpedia.org/resource/Johnny_Depp>

SPARQL Examples (ii)

Films starring “Johnny Depp”

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT ?m WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
}
```

(*) `dbo:starring` comes from the `https://dbpedia.org/ontology/`

SPARQL Examples (ii)

Films starring “Johnny Depp”

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT ?m WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
}
```

Answer:

?m
<http://dbpedia.org/resource/Dead_Man>
<http://dbpedia.org/resource/Edward_Scissorhands>
<http://dbpedia.org/resource/Arizona_Dream> ...

(*) `dbo:starring` comes from the `https://dbpedia.org/ontology/`

SPARQL Examples (iii)

Names of people who co-starred with “Johnny Depp”

```
SELECT DISTINCT ?costar WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
}
```

SPARQL Examples (iii)

Names of people who co-starred with “Johnny Depp”

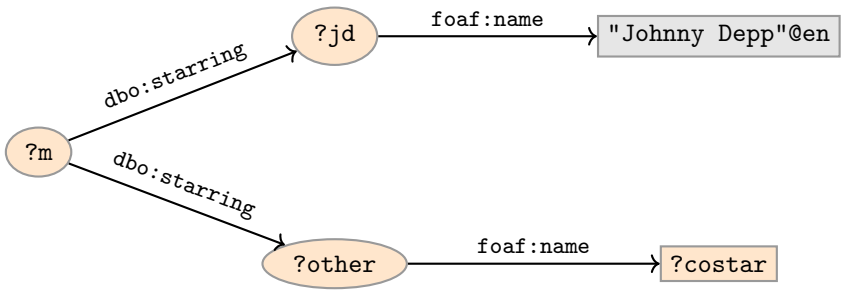
```
SELECT DISTINCT ?costar WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
}
```

Answer:

?costar
"Al Pacino"@en
"Antonio Banderas"@en
"Johnny Depp"@en
"Marlon Brando"@en
...

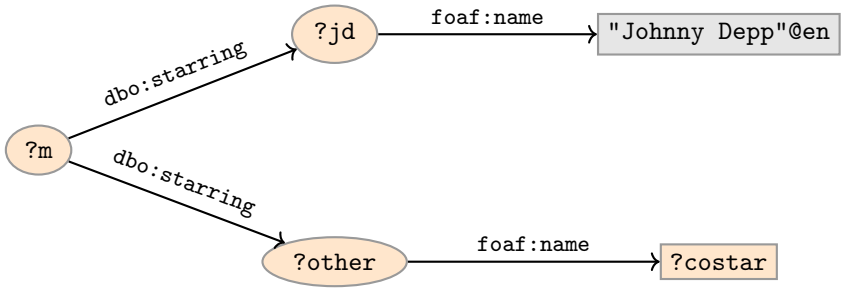
Graph Patterns

The previous SPARQL query as a graph:



Graph Patterns

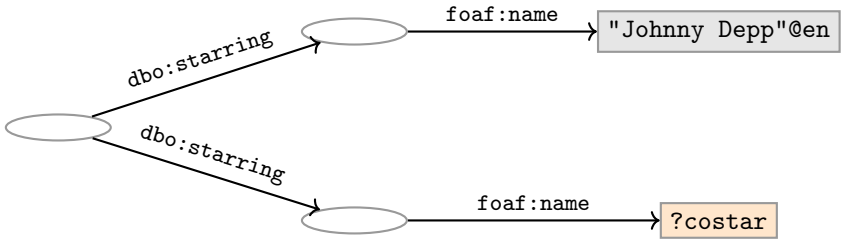
The previous SPARQL query as a graph:



Pattern matching: assign values to variables to make this a sub-graph of the RDF graph!

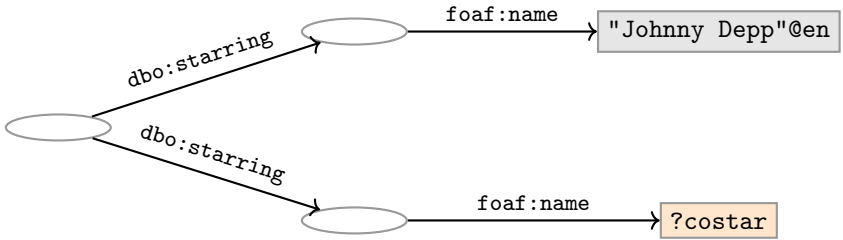
Graph with blank nodes

Variables not SELECTed can equivalently be blank:



Graph with blank nodes

Variables not SELECTed can equivalently be blank:



Pattern matching: a function that assigns values (*i.e.*, resource, a blank node, or a literal) to variables and blank nodes to make this a sub-graph of the RDF graph!

SPARQL Systematically

Components of a SPARQL query

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10
```

Components of a SPARQL query

Prologue: prefix definitions

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10
```

Components of a SPARQL query

Results: (1) query type (SELECT, ASK, CONSTRUCT, DESCRIBE), (2) remove duplicates (DISTINCT, REDUCED), (3) variable list.

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10
```

Components of a SPARQL query

Query pattern: graph pattern to be matched

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10
```

Components of a SPARQL query

Solution modifiers: ORDER BY, LIMIT, OFFSET

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10
```

Types of Queries (i)

SELECT Compute table of bindings for variables

```
SELECT DISTINCT ?a ?b WHERE {
  [ dbo:starring ?a ;
    dbo:starring ?b ]
}
```

Types of Queries (i)

SELECT Compute table of bindings for variables

```
SELECT DISTINCT ?a ?b WHERE {
  [ dbo:starring ?a ;
    dbo:starring ?b ]
}
```

CONSTRUCT Use bindings to construct a new RDF graph

```
CONSTRUCT {
  ?a foaf:knows ?b .
} WHERE {
  [ dbo:starring ?a ;
    dbo:starring ?b ]
}
```

Types of Queries (ii)

ASK Answer (yes/no) whether there is ≥ 1 match

```
ASK WHERE {
    ?jd foaf:name "Johnny Depp"@en .
}
```

Types of Queries (ii)

ASK Answer (yes/no) whether there is ≥ 1 match

```
ASK WHERE {
    ?jd foaf:name "Johnny Depp"@en .
}
```

DESCRIBE Returns an RDF graph with data about matching resources

```
DESCRIBE ?jd WHERE {
    ?jd foaf:name "Johnny Depp"@en .
}
```

SPARQL Systematically: Solution Modifiers

Solution Sequences and Modifiers

- Permitted to SELECT queries only
- SELECT treats solutions as a sequence (**solution sequence**)
- Query patterns generate an **unordered collection** of solutions
- **Sequence modifiers** can modify the solution sequence (not the solution itself). Applied in this order:
  - Order
  - Projection
  - Distinct
  - Reduced
  - Offset
  - Limit

ORDER BY

- Used to sort the solution sequence in a given way:
- `SELECT ... WHERE ... ORDER BY ...`
- `ASC` for ascending order (default) and `DESC` for descending order

ORDER BY

- Used to sort the solution sequence in a given way:
- `SELECT ... WHERE ... ORDER BY ...`
- `ASC` for ascending order (default) and `DESC` for descending order
- E.g.

```
SELECT ?city ?pop WHERE {
  ?city dbo:country ?country ;
        dbo:populationUrban ?pop .
} ORDER BY ?country DESC(?pop)
```

ORDER BY

- Used to sort the solution sequence in a given way:
- `SELECT ... WHERE ... ORDER BY ...`
- `ASC` for ascending order (default) and `DESC` for descending order
- E.g.

```
SELECT ?city ?pop WHERE {
  ?city dbo:country ?country ;
        dbo:populationUrban ?pop .
} ORDER BY ?country DESC(?pop)
```

- Standard defines **sorting conventions** for literals, URIs, etc.
- Not all “sorting” variables are required to appear in the `SELECT`ion.

ORDER BY (Example)

```
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
```

Projection, DISTINCT, REDUCED

- **Projection** (*i.e.*, `SELECT`ed variables) means that only some variables are part of the solution
  - Done with `SELECT ?x ?y WHERE {?x dbo:starring ?y . }`

Projection, DISTINCT, REDUCED

- **Projection** (*i.e.*, `SELECT`ed variables) means that only some variables are part of the solution
  - Done with `SELECT ?x ?y WHERE {?x dbo:starring ?y . }`
- DISTINCT **eliminates (all) duplicate** solutions:
  - Done with `SELECT DISTINCT ?x ?y WHERE {?x dbo:starring ?y. }`
  - A solution is a duplicate if it assigns the **same RDF terms to all variables** as another solution.

Projection, DISTINCT, REDUCED

- **Projection** (*i.e.*, `SELECT`ed variables) means that only some variables are part of the solution
  - Done with `SELECT ?x ?y WHERE {?x dbo:starring ?y . }`
- DISTINCT **eliminates (all) duplicate** solutions:
  - Done with `SELECT DISTINCT ?x ?y WHERE {?x dbo:starring ?y. }`
  - A solution is a duplicate if it assigns the **same RDF terms to all variables** as another solution.
- REDUCED allows to **remove some** or all duplicate solutions
  - Done with `SELECT REDUCED ?x ?y WHERE {?x dbo:starring ?y . }`
  - Motivation: Can be expensive to find and remove all duplicates
  - Behaviour left to the SPARQL engine.

OFFSET and LIMIT

- LIMIT: limits the number of results
- OFFSET: position/index of the first returned result
- Useful for paging through a large set of solutions

OFFSET and LIMIT

- LIMIT: limits the number of results
- OFFSET: position/index of the first returned result
- Useful for paging through a large set of solutions
- For example, solutions number 51 to 60:

```
SELECT ?x ?y  WHERE {?x dbo:starring ?y .} ORDER BY ?x
LIMIT 10 OFFSET 50
```

OFFSET and LIMIT

- LIMIT: limits the number of results
- OFFSET: position/index of the first returned result
- Useful for paging through a large set of solutions
- For example, solutions number 51 to 60:

```
SELECT ?x ?y  WHERE {?x dbo:starring ?y .} ORDER BY ?x
LIMIT 10 OFFSET 50
```

- LIMIT and OFFSET can be used separately
- `OFFSET` not meaningful without `ORDER BY`.

OFFSET and LIMIT (Example)

```
SELECT DISTINCT ?costar
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10 OFFSET 50
```

SPARQL Systematically: Query Graph Patterns

Query patterns

- Types of graph patterns for the query pattern (**WHERE clause**):
  - ✓ Basic Graph Patterns (BGP)
  - Filters or Constraints (FILTER)
  - Optional Graph Patterns (OPTIONAL)
  - Union Graph Patterns (UNION, Matching Alternatives)
  - Graph Graph Patterns (RDF Datasets)

Filters (i)

- A set of triple patterns may include **constraints** or **filters**
- Reduces matches of surrounding group where filter applies
- Example:

```
SELECT ?x
WHERE {
  ?x a dbo:Place ;
     dbo:populationUrban ?pop .
  FILTER (?pop > 1000000)
}
```

Filters (ii)

– Example:

```
SELECT DISTINCT ?costar
FROM <http://dbpedia_dataset>
WHERE {
    ?jd foaf:name "Johnny Depp"@en .
    ?m dbo:starring ?jd .
    ?m dbo:starring ?other .
    ?other foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
LIMIT 10 OFFSET 50
```

Filters: Functions and Operators

- Usual binary operators: `||`, `&&`, `=`, `!=`, `<`, `>`, `<=`, `>=`, `+`, `-`, `*`, `/`.
- Usual unary operators: `!`, `+`, `-`.
- Unary tests: `bound(?var)`, `isURI(?var)`, `isBlank(?var)`, `isLiteral(?var)`.
- Accessors: `str(?var)`, `lang(?var)`, `datatype(?var)`, `year(?date)`, `xsd:integer(?value)`
- `regex` is used to match a variable with a regular expression. *Always use with* `str(?var)`. E.g.: `regex(str(?costar), "Alpacino")`.

More details in specification: `http://www.w3.org/TR/rdf-sparql-query/`

OPTIONAL Patterns

– Allows a match to leave some variables **unbound** (e.g. no data is available). *e.g.*,:

```
WHERE {
  ?x a dbo:Person ;
     foaf:name ?name .
  OPTIONAL {
    ?x dbo:birthDate ?date .
  }
}
```

OPTIONAL Patterns

– Allows a match to leave some variables **unbound** (e.g. no data is available). *e.g.*,:

```
WHERE {
  ?x a dbo:Person ;
     foaf:name ?name .
  OPTIONAL {
    ?x dbo:birthDate ?date .
  }
}
```

– `?x` and `?name` bound in every match, `?date` is **bound if available**.

OPTIONAL Patterns

- Allows a match to leave some variables **unbound** (e.g. no data is available). *e.g.*,:

```
WHERE {
  ?x a dbo:Person ;
     foaf:name ?name .
  OPTIONAL {
    ?x dbo:birthDate ?date .
  }
}
```

- `?x` and `?name` bound in every match, `?date` is **bound if available**.
- Groups can contain several **optional parts**, evaluated separately

OPTIONAL Patterns: with FILTER

– Example:

```
WHERE {
  ?x a dbo:Person ;
     foaf:name ?name .
  OPTIONAL {
    ?x dbo:birthDate ?date .
    FILTER (?date > "1980-01-01T00:00:00"^^xsd:dateTime)
  }
}
```

– `?x` and `?name` bound in every match, `?date` is **bound if available** and **from 1980 onwards**.

Matching Alternatives (UNION)

- A UNION pattern matches if any of some alternatives matches
- E.g.

```
SELECT DISTINCT ?writer
WHERE{
  ?s rdf:type dbo:Book .
  {
    ?s dbo:author ?writer .
  }
  UNION
  {
    ?s dbo:writer ?writer .
  }
}
```

SPARQL over Named Graphs

Recap: 'Graph' Graph Patterns (RDF datasets)

- SPARQL queries are executed against an **RDF dataset**
- An RDF dataset comprises
  - One **default graph** (unnamed) graph.
  - Zero or more **named graphs** identified by an URI

Recap: 'Graph' Graph Patterns (RDF datasets)

- SPARQL queries are executed against an **RDF dataset**
- An RDF dataset comprises
  - One **default graph** (unnamed) graph.
  - Zero or more **named graphs** identified by an URI
- FROM and FROM NAMED keywords allows to select an RDF dataset
- Keyword GRAPH makes the named graphs the **active graph** for pattern matching

Recap: 'Graph' Graph Patterns (RDF datasets)

- SPARQL queries are executed against an **RDF dataset**
- An RDF dataset comprises
  - One **default graph** (unnamed) graph.
  - Zero or more **named graphs** identified by an URI
- FROM and FROM NAMED keywords allows to select an RDF dataset
- Keyword GRAPH makes the named graphs the **active graph** for pattern matching
- Not well supported in Jena and rdflib. Supported in GraphDB (lab).

‘Graph’ Graph Patterns: examples (i)

```
SELECT DISTINCT ?person
WHERE {
  GRAPH city:academic-year-2022 {
    ?person rdf:type foaf:Person .
  }
}
```

‘Graph’ Graph Patterns: examples (ii)

```
SELECT DISTINCT ?person
WHERE {
  GRAPH ?named_graph {
    ?person rdf:type foaf:Person .
  }
}
```

‘Graph’ Graph Patterns: examples (iii)

```
SELECT DISTINCT ?person
FROM city:academic-year-2021
FROM city:academic-year-2022
{
  ?person rdf:type foaf:Person .
}
```

SPARQL 1.1

SPARQL 1.1: new features

- The new features in **SPARQL 1.1 QUERY language**:
  - Assignments and Expressions
  - Aggregates
  - Subqueries
  - Negation (new syntax)
  - Property paths

SPARQL 1.1: new features

- The new features in **SPARQL 1.1 QUERY language**:
  - Assignments and Expressions
  - Aggregates
  - Subqueries
  - Negation (new syntax)
  - Property paths
- Specification for:
  - **SPARQL 1.1 UPDATE Language**
  - **SPARQL 1.1 Federated Queries**
  - **SPARQL 1.1 Entailment Regimes**

Assignment and Expressions

- The value of an expression can be assigned/bound to a new variable
- Can be used in SELECT, BIND or GROUP BY clauses:
  (expression AS ?var)

Expressions in SELECT clause

```
SELECT ?city (xsd:integer(?pop)/xsd:float(?area) AS ?density)
{
  ?city dbo:populationTotal ?pop .
  ?city dbo:PopulatedPlace/areaTotal ?area .
  ?city dbo:country dbr:United_Kingdom .
  FILTER (xsd:float(?area)>0.0)
}
```

Aggregates: Grouping and Filtering

- Solutions can optionally be grouped according to one or more expressions.
- To specify the group, use `GROUP BY`.
- If `GROUP BY` is not used, then **only one (implicit) group**

Aggregates: Grouping and Filtering

- Solutions can optionally be grouped according to one or more expressions.
- To specify the group, use `GROUP BY`.
- If `GROUP BY` is not used, then **only one (implicit) group**
- To filter solutions resulting from grouping, use `HAVING`.
- `HAVING` operates over grouped solution sets, in the same way that FILTER operates over un-grouped ones.

Aggregates: Example

Actors with more than 15 movies

```
SELECT ?name (COUNT(?movie) AS ?mcount)
WHERE {
    ?actor foaf:name ?name .
    ?movie dbo:starring ?actor .
}
GROUP BY ?name
HAVING (COUNT(?movie) > 15)
ORDER BY DESC (?mcount)
```

Aggregates: Example

Actors with more than 15 movies

```
SELECT ?name (COUNT(?movie) AS ?mcount)
WHERE {
    ?actor foaf:name ?name .
    ?movie dbo:starring ?actor .
}
GROUP BY ?name
HAVING (COUNT(?movie) > 15)
ORDER BY DESC (?mcount)
```

† Only expressions consisting of aggregates and constants may be projected, together with variables in `GROUP BY`.

Aggregates: common functions

- `Count` counts the number of times a variable has been bound.
- `Sum` sums numerical values of bound variables.
- `Avg` finds the average of numerical values of bound variables.
- `Min` finds the minimum of the numerical values of bound variables.
- `Max` finds the maximum of the numerical values of bound variables.

† Aggregates assume CWA and UNA

Subqueries

- A way to embed SPARQL queries within other queries
- Subqueries are evaluated first and the results are projected to the outer query.

Subqueries

- A way to embed SPARQL queries within other queries
- Subqueries are evaluated first and the results are projected to the outer query.

```
SELECT ?country ?pop (round(?pop/?worldpop*1000)/10 AS ?percentage) WHERE {
  ?country rdf:type dbo:Country .
  ?country dbo:populationTotal ?pop .
  {
    SELECT (sum(?p) AS ?worldpop) WHERE {
      ?c rdf:type dbo:Country .
      ?c dbo:populationTotal ?p .}
  }
}  ORDER BY desc(?pop)
```

Subqueries

- A way to embed SPARQL queries within other queries
- Subqueries are evaluated first and the results are projected to the outer query.

```
SELECT ?country ?pop (round(?pop/?worldpop*1000)/10 AS ?percentage) WHERE {
  ?country rdf:type dbo:Country .
  ?country dbo:populationTotal ?pop .
  {
    SELECT (sum(?p) AS ?worldpop) WHERE {
      ?c rdf:type dbo:Country .
      ?c dbo:populationTotal ?p .}
  }
}  ORDER BY desc(?pop)
```

† Note that the *Sum()* aggregation is done over all the elements (single default group).

Negation in SPARQL 1.1: MINUS and FILTER NOT EXISTS

Two ways to do negation. *e.g.*, retrieve people without a name:

```
SELECT DISTINCT * WHERE {
    ?person a foaf:Person .
    MINUS { ?person foaf:name ?name }
}

SELECT DISTINCT * WHERE {
    ?person a foaf:Person .
    FILTER NOT EXISTS { ?person foaf:name ?name }
}
```

Property paths: basic motivation

- Some queries get needlessly large.
- SPARQL 1.1 define a small language to defined paths.
- Examples:
  - `city:ernesto foaf:knows+ ?friend` to extract all friends of friends.
  - `foaf:maker|dc:creator` instead of `UNION`.
  - Friend's names, `{ _:me foaf:knows/foaf:name ?friendsname }`.
  - Sum several items:

```
SELECT (sum(?cost) AS ?total) { :order :hasItem/:price ?cost }
```

Property paths: example

```
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dbo:  <http://dbpedia.org/ontology/>
SELECT DISTINCT ?costar
WHERE {
    ?m dbo:starring/foaf:name "Johnny Depp"@en .
    ?m dbo:starring/foaf:name ?costar .
    FILTER (STR(?costar)!="Johnny Depp")
}
ORDER BY ?costar
```

†Similar to blank node syntax.

Property paths: syntax

Syntax Form	Matches
`iri`	An (property) IRI. A path of length one.
`^elt`	Inverse path (object to subject).
`elt1 / elt2`	A sequence path of `elt1` followed by `elt2`.
`elt1 | elt2`	A alternative path of `elt1` or `elt2` (all possibilities are tried).
`elt*`	Seq. of zero or more matches of `elt`.
`elt+`	Seq. of one or more matches of `elt`.
`elt?`	Zero or one matches of `elt`.
`!iri or !(`$\texttt{iri}_1$`| ...|`$\texttt{iri}_n$`)`	Negated property set.
`!^iri or !(^`$\texttt{iri}_i$`| ...|^`$\texttt{iri}_n$`)`	Negation of inverse path.
`!(`$\texttt{iri}_1$`|...|`$\texttt{iri}_j$`|^`$\texttt{iri}_{j+1}$`|...|^`$\texttt{iri}_n$`)`	Negated combination of forward and inverese properties.
`(elt)`	A group path `elt`, brackets control precedence.

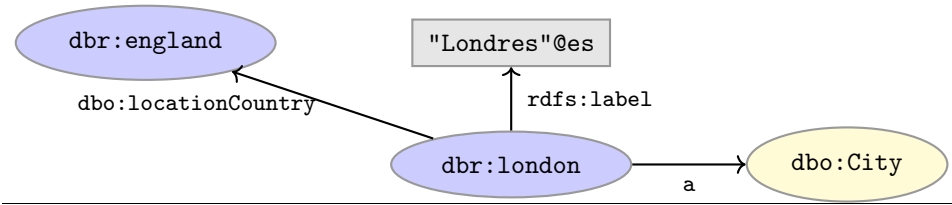
* `elt` is a path element, which may itself be composed of path constructs (see Syntax form).

SPARQL Summary

SPARQL Summary (i)

Return all Cities:

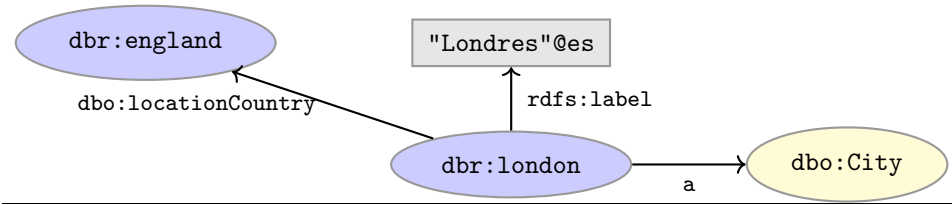
```
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
SELECT DISTINCT ?city WHERE {
    ?city rdf:type dbo:City .
}
```



SPARQL Summary (i)

Return all Cities: Query Result= {dbr:london}

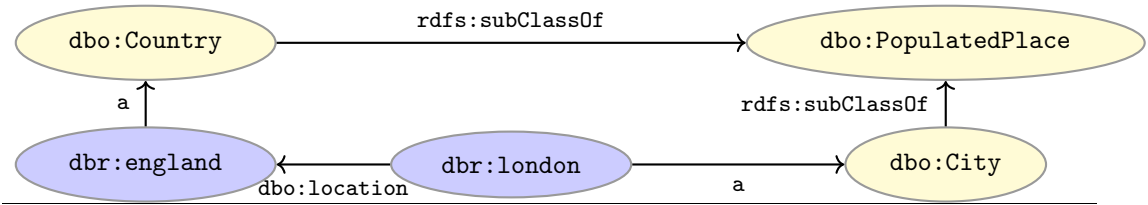
```
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
SELECT DISTINCT ?city WHERE {
    ?city rdf:type dbo:City .
}
```



SPARQL Summary (ii)

Return all Populated Places:

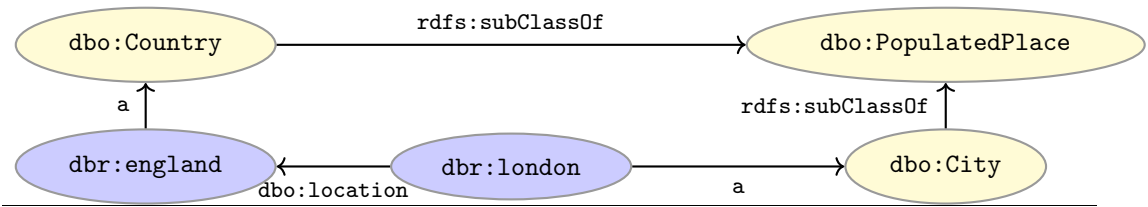
```
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
SELECT DISTINCT ?place WHERE {
    ?place rdf:type dbo:PopulatedPlace .
}
```



SPARQL Summary (ii)

Return all Populated Places: Query Result= {}

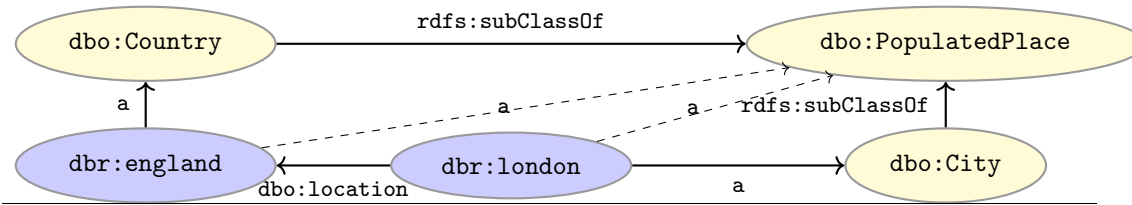
```
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
SELECT DISTINCT ?place WHERE {
    ?place rdf:type dbo:PopulatedPlace .
}
```



SPARQL Summary (iii) with reasoning!

Query Result= {dbr:england, dbr:london}

```
PREFIX dbo:  <http://dbpedia.org/ontology/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
SELECT DISTINCT ?place WHERE {
    ?place rdf:type dbo:PopulatedPlace .
}
```



Graph Database Solutions

How to store Graph models?

- Relational model
  - Single table with 3 columns (source, edge, target)
  - Property tables (one table per type of entity, *e.g.*, Person, Module)
  - Binary tables (one table per relationship, *e.g.*, source, target)

M. Wylot and others. RDF Data Storage and Query Processing Schemes. ACM Computing Surveys 2018

How to store Graph models?

- Relational model
  - Single table with 3 columns (source, edge, target)
  - Property tables (one table per type of entity, *e.g.*, Person, Module)
  - Binary tables (one table per relationship, *e.g.*, source, target)
- **Graph-based databases** (NoSQL)

M. Wylot and others. RDF Data Storage and Query Processing Schemes. ACM Computing Surveys 2018

Introduction

Advanced graph database solutions:

- Scale to large Knowledge Graphs.
- Sophisticated indexing structures.
- Optimised reasoning.
- Fast query performance.
- Server solution in production.

State-of-the-art solutions

Dimensions:

- Free version.
- Compliance with Semantic Web standards.
- Reasoning capabilities.
- In-memory or In-disk.
- Documentation and installation requirements.
- Additional features.

A Survey of RDF Stores & SPARQL Engines for Querying Knowledge Graphs. arXiv:2102.13027 2021 (Appendix A)

Semantic Web standards

- The **World Wide Web Consortium (W3C)** is an international community that develops open standards to ensure the long-term growth of the Web: `https://www.w3.org/`
- On the Web and **beyond**.
- **Why standards?**
  - broader industry (and academic) agreement,
  - interoperability across organizations and applications,
  - avoids vendor lock-in of a particular (exchange or query) format.

Apache Jena TDB

- Free solution.
- Provides a native (in-disk) RDF store.
- In combination with Jena Fuseki to provide SPARQL Endpoint support.
- ✗ Supports reasoning as in Jena, but not direct support for OWL 2 nor the OWL 2 profiles.

`https://jena.apache.org/documentation/tdb/`

OpenLink Virtuoso

- ✓ Provides the SPARQL endpoint for DBpedia.
- – Open source and commercial versions.
- – Object-oriented database model.
- ✓ Native graph model storage provider for Jena and RDF4J.
- ✗ Custom inference rules. Partial support for OWL 2.

`https://virtuoso.openlinksw.com/`

`http://vos.openlinksw.com/owiki/wiki/VOS`

Blazegraph

- ✓ Provides the SPARQL Endpoint for Wikidata.
- ✓ Free and open source.
- – Both in-memory and disk-oriented storage.
- ✗ Only supports OWL 1 Lite reasoning.
- – Blazegraph team now working for Amazon.

`https://blazegraph.com/`

AllegroGraph

– Free and commercial licenses.

✗ Supports only RDFS++ materialization.

✓ Client interface in several languages.

– Can be used to query both documents and graph data (via SPARQL).

`https://allegrograph.com/`

Neo4j

- ✓ Open source graph database.
- – Based on the Property Graph Model.
- ✗ Cypher as graph query language (no native SPARQL support).
- – Support *via a plugin* for RDF, RDFS and OWL vocabularies.
- ✗ Basic inferencing support.
- ✓ Support for Analytics.
- ✓ Interfaces in many languages.

`https://neo4j.com/`

TypeDB (formerly GRAKN.AI)

✓ TypeDB is an open-source, distributed knowledge graph database.

✓ Support for analytics.

✓ Interesting integration with machine learning models.

– Provides inferencing support.

✗ No support for any of the Semantic Web standards.

`https://vaticle.com/`
GRAKN.AI vs Semantic Web standards (some justifications that I do not personally share):
`https://blog.grakn.ai/knowledge-graph-representation-grakn-ai-or-owl-506065bd3f24`

GraphDB (formerly OWLIM)

– Free and commercial versions.

✓ Very easy to install and use.

✓ Powerful reasoning features: including OWL 2 QL and RL profiles.

✓ Supports SHACL validation.

– Includes text indexing via lucene.

– Powered the early Linked Data services at the BBC.

✓ **Our choice for the lab today.**

`https://www.ontotext.com/products/graphdb/`

RDF4J (formerly Sesame)

✓ General Java framework to manage RDF data.

– Provides native (in-memory and in-disk) storage solutions.

– Can connect to other RDF stores *e.g.*,: Blazegraph, Amazon Neptune, GraphDB, Virtuoso.

– Indexing, reasoning and query processing techniques depend on the underlying storage engine.

`https://rdf4j.org/`

RDFox

- – Commercial system. Free academic license on request.
- ✓ Support for materialization-based datalog reasoning (including OWL 2 RL and SWRL rules).
- ✓ Supports SHACL validation.
- – In-memory RDF engine.
- ✓ Access via Java API or remotely via REST API or SPARQL Endpoint.
- ✗ Limitation on the size of the memory.
- ✓ **Our choice for the lab today.**

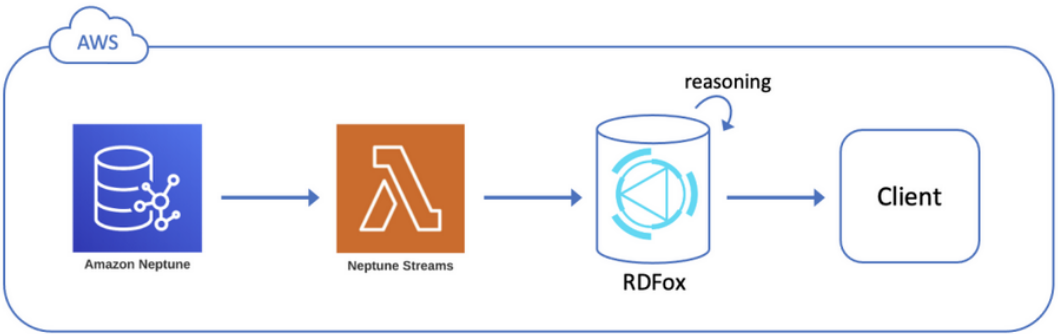
`https://www.oxfordsemantic.tech/product`

Amazon Neptune

- Cloud-based only solution (graph database as a service!).
- Blazegraph is now part of Amazon Neptune.
- On-Demand pricing
- ✓ Support for RDF.
- ✓ Support for both OpenCypher and SPARQL query languages.
- ✗ Native inferencing is not yet supported.
- ✓ Keep an eye on future development!

`https://aws.amazon.com/neptune/`

Amazon Neptune + RDFox



Amazon Neptune and RDFox: `https://aws.amazon.com/blogs/database/use-semantic-reasoning-to-infer-new-facts-from-your-rdf-graph-by-integrating-rdfox-with-amazon-neptune/`

Graph database/Triplestore benchmarking

- Oracle Database 12c: 1.08 Trillion triples
- AnzoGraph DB: 1.06 Trillion triples
- AllegroGraph: 1.0 Trillion triples
- Virtuoso: 94.2 Billion Triples
- Stardog: 50 Billion triples
- **RDFox**: 19.47 Billion triples
- **GraphDB**: 17 Billion triples
- Apache Jena TBD: 16.7 billion triples.

Numbers may not be up-to-date: `https://www.w3.org/wiki/LargeTripleStores`

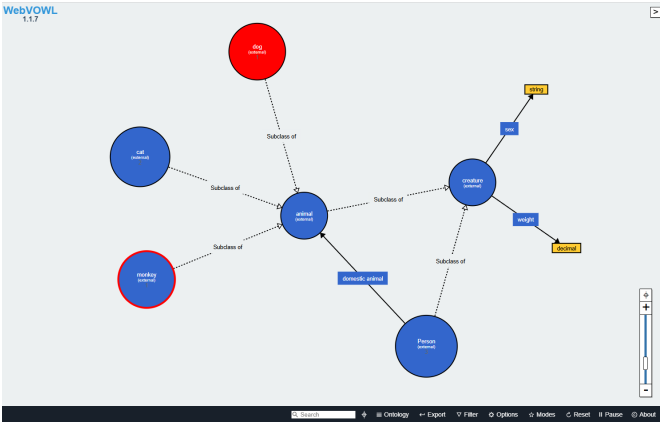
Visualization

Example of available tools

- `Protégé` provides a visual abstraction via hierarchy and axiom views. It also provides some plugin support to visualise the ontology as a graph.
- Graph database solutions like **GraphDB** also allow exploring the KG via a visual graph *(lab)*.
- Larger KGs may need to be visualised using different techniques, *e.g.*, knowledge graph embeddings *(session 6)*.
- **WebVOWL** is a prominent example of a tool to visualise OWL ontologies *(lab)*.

Marek Dudás, Steffen Lohmann, Vojtech Svátek, Dmitry Pavlov: Ontology visualization methods and tools: a survey of the state of the art. Knowl. Eng. Rev. 33: e10 (2018)
Web-VOWL: `https://github.com/VisualDataWeb/WebVOWL`

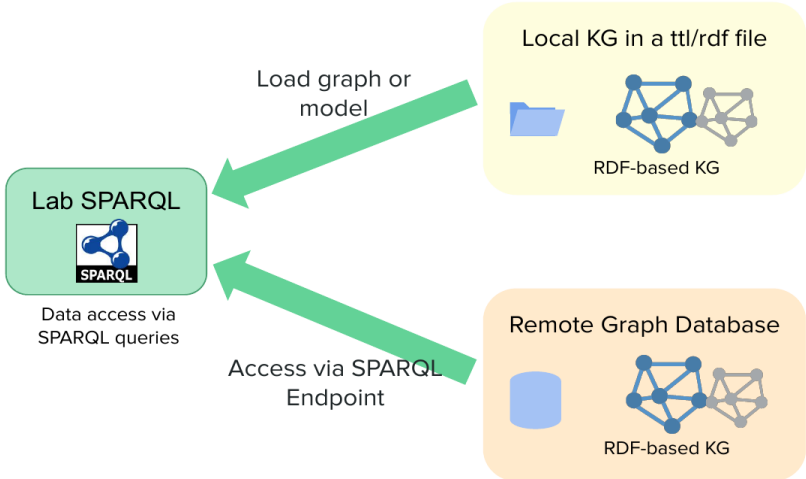
WebVOWL: Visualisation of OWL ontologies



WebVOWL: `https://github.com/VisualDataWeb/WebVOWL`

Laboratory: Hands-on with SPARQL and Graph Databases

Laboratory: SPARQL over local and remote KGs



Laboratory: Graph Databases

- Using the KG store **GraphDB**
  - Creating repositories.
  - Loading data and ontology.
  - Querying the data
  - Visualising the data